PLANNING OF ELECTRICITY PRODUCTION AND TRANSMISSION

Project Task: Nonlinear optimization

Abstract

This project focuses on the development of a simplified model for the power supply network in Sweden, aimed at optimizing the balance between power generation and consumption while minimizing active power production costs. By incorporating crucial parameters such as voltage amplitudes, phase angles, and power flows, the model provides a comprehensive view of the dynamics in power generation and transmission. Through rigorous numerical optimization, the project offers valuable insights into efficient power supply management applicable in real-world contexts.

Omkar Parishwad, Sanjaya Thilak

omkarp@chalmers.se

Contents

Introduction 1
Mathematical Formulation 3
Parameters 4
Decision Variables 4
Objective Function 4
Constraints 5
1. Variable bounds 5
2. Power Flow Equations 5
3. Power Conservation Constraints 5
4. Generation and Load Balance Constraints 6
Results and analysis 7
Optimality 9
Sensitivity analysis 9
Summary and conclusions 10

Appendix A: Julia/JuMP Code 11
Appendix B: Code Output: 16
Appendix C: Results (2D DenseAxisArray) 22

List of Tables

Table 1. Active and reactive power produced by each generator 7
Table 2. The voltage amplitudes and the voltage phase angle for the nodes 8
Table 3. Active and reactive power flowing along the network's edges 8
Table 4. Active power matrix for nodes (11x11) 22
Table 5. Reactive power matrix for nodes (11x11) 23

Introduction

Power supply dynamics in intricate networks like those found in Sweden can be complex and challenging to understand. This project aims to optimize these interactions through a streamlined model that encapsulates the critical aspects of power generation and transmission. The simplified map developed in this study assists in understanding the mathematical model we propose, making it accessible even for those unfamiliar with the project.

A fundamental aspect of any power network is the balance between reactive and active power. Reactive power is essential for maintaining the voltage levels required to transmit active power, which performs actual work, across the network. The primary focus of this study is to optimize the production and distribution of active power to meet demands cost-effectively.

The proposed model considers vital parameters such as voltage amplitudes, phase angles, and power flows between network nodes. These parameters serve as the guiding metrics for a balanced and efficient power supply, ensuring grid stability and operational efficiency. They also play a crucial role in minimizing energy losses and addressing issues related to non-symmetry in power flow.

Decision variables, intrinsically linked to the physical components within the network, are at the core of our model. These variables are pivotal in achieving our primary objective of minimizing active power production costs and determining the overall cost of power supply. A carefully constructed set of constraints helps us navigate the complexities associated with power flow and generation, ultimately guiding us toward an optimal solution.

Overall, this project provides a clear pathway to an ideal solution for efficient power generation and consumption balance. The insights gained are, in concept, applicable to the Swedish power network as well as for a broader, real-world context.

Mathematical Formulation

The optimization problem addressed in this report focuses on minimizing the cost associated with the production of active power within a complex power generation and distribution network. The primary objective is to ensure that the consumers' demand for active power is met efficiently while minimizing operational costs.

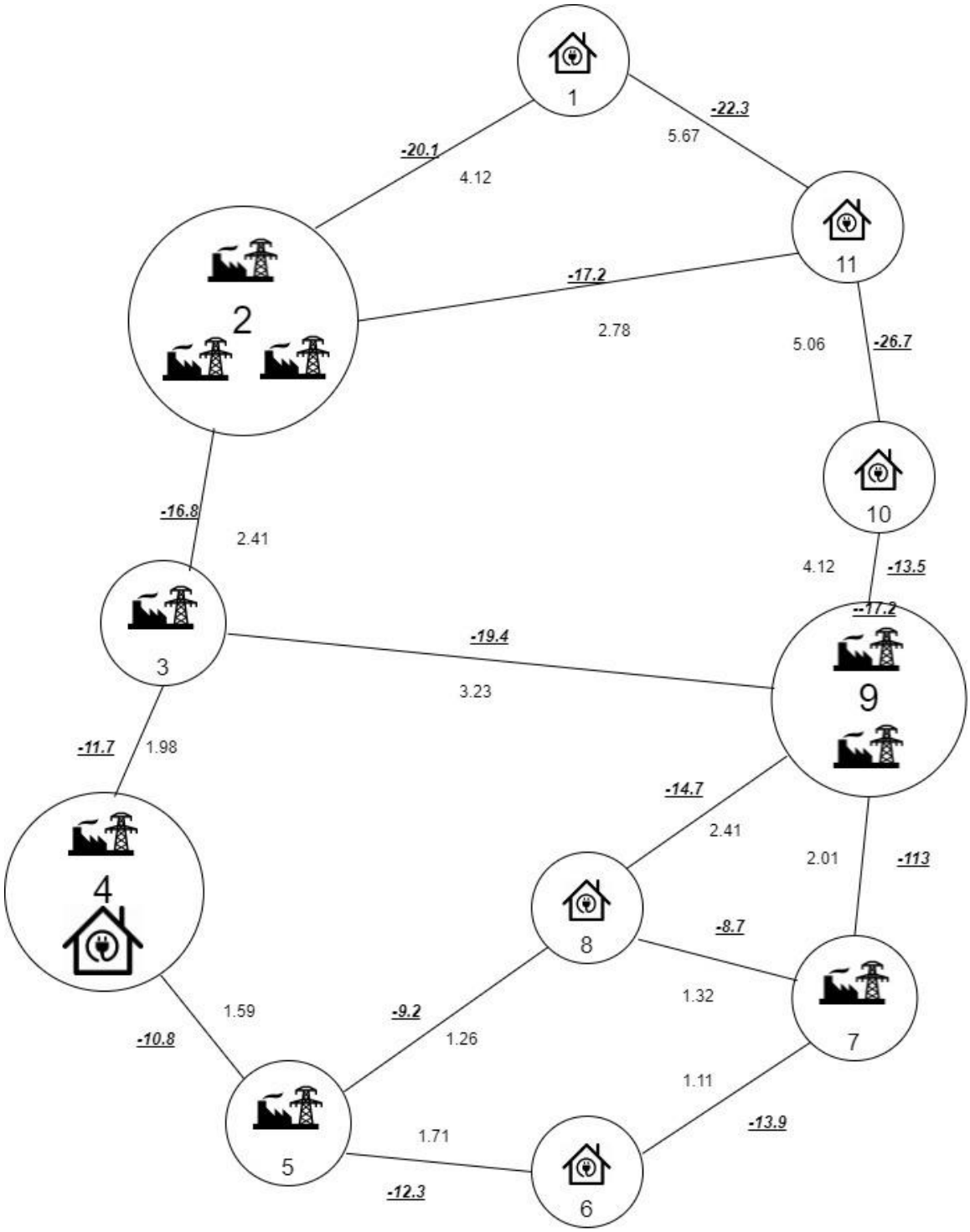

Figure 1. Power flows as defined for optimization model

Parameters

The optimization model is defined using the following parameters:

- $cost[i]$: The energy production cost [SEK/pu] associated with generator i. These values represent the cost per unit of active power generation for each generator.
- $max_capacity[i]$: The maximum capacity [pu] of generator i. These values define the upper bounds on active power generation for each generator.
- $demand[i]$: The consumer demand [pu] for active power at node i. These values represent the active power required at each node in the network.
- $g[k,l]$: The conductance (real part of admittance) between nodes k and l. These values define the conductance between network nodes and play a role in power flow calculations.
- $b[k,l]$: The susceptance (imaginary part of admittance) between nodes k and l. These values represent the susceptance between network nodes and contribute to power flow calculations.
- $GENERATORINDEX$: Generators indexed {1,2,..,9} for context.
- $NODEINDEX$: Distribution Nodes indexed {1,2,..,11} for context.

Decision Variables

- $active[i]$: Active power generation of generator i.
- $reactive[i]$: Reactive power generation or absorption of generator i.
- $v[i]$: Voltage magnitude at node i.
- $theta[i]$: Voltage phase angle at node i.
- $active_power_flow[k,l]$: The active power flow in each edge of the transmission grid network
- $reactive_power_flow[k,l]$: The reactive power flow in each edge of the transmission grid network.

Objective Function

The objective of this optimization problem is to minimize the total cost of active power production. This is represented by the following objective function.

$$\text{Minimize: } \sum_{i \in GENERATORINDEX} * cost[i] * active[i]$$

Constraints

1. **Variable bounds**
   - **Voltage Magnitude and Phase Angle Constraints**

     For each node i, voltage magnitude $v[i]$ and voltage phase angle $theta[i]$ are bounded are the following. These constraints ensure that magnitudes and phase angles remain within acceptable limits.

$$0.98 \leq v[i] \leq 1.02$$

$$-3.14159 \leq theta[i] \leq 3.14159$$

   - **Active and Reactive Power Bounds**

     For each generator i, the following bounds are imposed on active and reactive power generation.

$$0 \leq active[i] \leq max_capacity[i]$$

$$-0.003 \,.max_capacity[i] \leq reactive[i] \leq 0.003 \,.max_capacity[i]$$

2. **Power Flow Equations**

   The power flow equations represent the active and reactive power flow between nodes k and l are the following. These equations describe the active and reactive power flows in the network.

$$p_var[k,l] = v[k]^2 \cdot g[k,l] - v[k] \cdot v[l] \cdot g[k,l] \cdot cos(\theta[k] - \theta[l]) - v[k] \cdot v[l] \cdot b[k,l] \cdot sin(\theta[k] - \theta[l])$$

$$q_var[k,l] = -v[k]^2 \cdot b[k,l] + v[k] \cdot v[l] \cdot b[k,l] \cdot cos(\theta[k] - \theta[l]) - v[k] \cdot v[l] \cdot g[k,l] \cdot sin(\theta[k] - \theta[l])$$

3. **Power Conservation Constraints**

   For each node k, the following power conservation equations ensure that the sum of active and reactive power injections equals the demand. These constraints enforce the balance of power at each node.

$$\sum_{i \in NODEINDEX} p_var[k,i] - G[k] + demand[k] = 0$$

$$\sum_{i \in NODEINDEX} q_var[k,i] - R[k] = 0$$

4. **Generation and Load Balance Constraints**

   The generation and load balance constraints ensure that power generation equals consumption at specific nodes. These constraints account for the power generation and consumption at each node in the network. For this, we define the active and reactive power generation location constraints, based on generator location, so that the summation of active power in the grid network, would be less than or equal to the demand for active power.

   The active power balance at each node i has been considered in the code as;

   $RG_1, RG_6, RG_8, RG_{10}, RG_{11} = 0$ ...(as these nodes do not have generators)

   $RG_3 = power_4$; $RG_4 = power_5$; $RG_5 = power_6$; $RG_7 = power_7$;

   $RG_9 = power_8 + power_9$; $RG_2 = power_1 + power_2 + power_3$

   Where, RG_i = active and reactive power generation

   $power_i$ = active and reactive power generated at generator i

   The active and reactive power generation for each node equated to the active and reactive power generated by each generator is considered separately (separate equations for active & reactive) as the balance constraints in the model.

These constraints ensure that at each node, the generated active and reactive power matches the sum of active and reactive power at connected generators, maintaining a balance between power generation and consumption.

The mathematical formulation presented here captures the essence of the optimization problem. It defines the decision variables, the objective function to minimize cost, and a set of constraints that ensure physical feasibility and power balance within the network. This formulation sets the stage for solving the optimization problem and obtaining valuable insights into power generation and distribution.

Results and analysis

Upon solving the optimization problem numerically using the Ipopt solver, we obtained a solution that minimizes the total cost of power production. The total cost is determined to be 186.2891 SEK. This represents the most cost-effective allocation of active power generation among the available generators while satisfying consumer demand.

The optimal solution provides these active and reactive power generations for each generator in the network (*at optimality), is as provided in Table 1. Generators 2, 3, 4, 5 and 9 are operating close to their maximum capacity, while generator 7 is the most under-utilised, suggesting that it may be used as a backup power generator. The reactive power seems fairly optimised to their lowest, but generator 8 has the highest comparative reactive power generation, indicating its greater contribution to voltage control.

Table 1. Active and reactive power produced by each generator

Generator	Max Capacity	Active power (pu)*	Reactive power (pu)*
1	0.02	0.004874	0.000060
2	**0.15**	0.150000	0.000450
3	**0.08**	0.080000	0.000240
4	**0.07**	0.070000	0.000210
5	**0.04**	0.040000	0.000120
6	0.17	0.138856	0.000510
7	0.17	*0.003079*	0.000510
8	0.26	0.253682	0.000780
9	**0.05**	0.050000	0.000150

The solution also provides the voltage characteristics- amplitude (v) and phase angle at each node, as defined at optimality, in Table 2. All the voltage levels seem within their acceptable range (near 1 pu) for voltage regulation. But it is lowest at node 4 with a negative phase angle, indicating minor concerns with load balancing. Small phase angle differences imply no significant voltage phase disparities between adjacent nodes, which can lead to power flow issues and instability.

Table 2. The voltage amplitudes and the voltage phase angle for the nodes

Node	Voltage magnitude (v)*	Phase angle (theta in radians)*
1	1.019132	0.001805
2	1.020000	0.006264
3	1.019487	0.003009
4	1.018143	-0.004844
5	1.018868	-0.000307
6	1.018213	-0.005299
7	1.018528	-0.002190
8	1.018516	-0.002560
9	1.019219	0.001567
10	1.019011	0.000642
11	1.019230	0.001921

The power flow analysis reveals the following active and reactive power flows along the transmission network's edges. These active and reactive power flow values, at optimality, are provided in Table 3 below.

Table 3. Active and reactive power flowing along the network's edges

Edge	Active power flow(pu)*	Reactive power flow(pu)*
2,1	**0.096766**	**-0.001102**
11,1	0.003234	0.001533
2,3	0.058139	0.000725
2,11	0.079881	**0.001128**
4,3	**0.098015**	0.000500
3,9	0.029966	0.000475
4,5	0.052019	-0.000380
6,5	0.064824	0.000808
5,8	0.021968	0.000379
6,7	0.045187	-0.000808
8,7	0.003359	0.000403

8,9	0.064724	-0.000074
10,9	0.013418	-0.000800
10,11	0.036591	0.000800

PS. The negative numbers here indicate the reversed direction of flow of power on edges.

Optimality

The set problem is a constrained optimization, so the obtained solution by the solver (Ipopt) is locally optimal, as it satisfies all constraints and represents the optimal allocation of active power generation to minimize cost. However, it's important to note that the global optimality of the solution depends on the problem's mathematical properties, i.e. it should satisfy the KKT conditions.

To ascertain global optimality, additional techniques like global optimization algorithms, sensitivity analysis, or mathematical analysis of the problem structure may be required. These methods aim to explore a wider solution space to identify the globally optimal solution, which may or may not coincide with a locally optimal solution, depending on the problem's characteristics.

Sensitivity analysis

To address the question of increasing the maximum capacity of one generator by 0.01 power unit without affecting reactive power generation, we can analyse the Lagrange multipliers associated with the active power generation bounds. By identifying the generator with the highest corresponding Lagrange multiplier associated with the active power generation bounds, we can determine which generator would be most beneficial to increase in maximum capacity.

However, since we have the simplification, that the increase would not affect the generator's ability to produce or absorb reactive power. Our first guess would be to check which generators are pushed to their maximum capacity limits (G2, G4. G5 and G9), and choose the one which has the lowest energy production cost (i.e. **G2**).

But, using Julia/JuMP, we can obtain the Lagrange multipliers for the active power generation bounds (constraint), by accessing the dual variables (shadow prices) corresponding to those constraints. They can help identify which generator's maximum capacity increase would have the most significant impact on the objective function (i.e., cost reduction) without violating constraints. Thus, the ***most beneficial generator for capacity increase would be Generator 5*** *(if we don't simplify)***.**

Summary and conclusions

We acquired valuable insights from this exercise about the optimizations and operations of the power generation and distribution network. We understand that usually, the power plants' capacity and production are converted to per-unit costs. We can gauge the amount of simplification that we undertake for this exercise. We also see that the simplified value of energy production costs may be related to the demand at the location.

The optimization model successfully allocates active power generation among the available generators in a cost-effective manner. The minimization of the total cost of active power production ensures that resources are utilized efficiently while meeting consumer demand. This not only reduces operational expenses but also contributes to the overall economic viability of the power network. Moreover, we can relate to the power losses as the electricity travels longer distances from the model values at optimality.
On the same note, voltage characteristics like magnitude and phase angle play a crucial role in ensuring the stability and reliability of the power system. The optimization results show that voltage magnitudes and phase angles are maintained within acceptable limits for all nodes in the network. This indicates that the proposed power flow and generation configuration maintains stable voltage profiles, minimizing voltage deviations and ensuring proper operation of connected loads.

Lastly, analysing active and reactive power flows along network edges provides a comprehensive view of the power transfer within the system. The highest active power flows occur between nodes 2 and 1, and nodes 4 and 3, with values around 0.1 pu. These edges likely represent critical transmission paths where significant power is being transmitted. The lowest active power flows occur on edges connected to nodes 11 and 8, indicating that these nodes are either lightly loaded or serve as endpoints for power transfer. Some edges, such as 2,1 and 2,11, exhibit a small amount of reactive power injection, indicating that these nodes are sources of reactive power. Reactive power is often injected to maintain voltage stability and control. Overall, the network maintains a balance between active and reactive power flows, as no extreme values are observed in either direction.

By examining these flows, we can identify congested or critical lines where power transfer may be approaching limits. Such insights are valuable for system operators in making informed decisions regarding network upgrades, maintenance, or adjustments to maintain power flow within desired limits.

Appendix A: Julia/JuMP Code

```
Project.jl file:

using JuMP
import Ipopt

# Import data from the data file
include("project_data.jl")

# Define the model
model = Model(Ipopt.Optimizer)

##########################################################
# Sets
##########################################################
NODEINDEX = 1:11
GENERATORINDEX = 1:9

##########################################################
# Variables
##########################################################

@variable(model, v[NODEINDEX])
@variable(model, theta[NODEINDEX])
@variable(model, active[GENERATORINDEX])
@variable(model, reactive[GENERATORINDEX])
@variable(model, G[NODEINDEX])
@variable(model, R[NODEINDEX])
@variable(model, p_var[NODEINDEX, NODEINDEX])
@variable(model, q_var[NODEINDEX, NODEINDEX])

##########################################################
# Objective function
##########################################################

@objective(model, Min, sum(cost[i] * active[i] for i in
GENERATORINDEX))

##########################################################
# Constraints
##########################################################
```

```
# Variable bounds
for i in NODEINDEX
    @constraint(model, 0.98 <= v[i] <= 1.02)
    @constraint(model, -3.14159 <= theta[i] <= 3.14159)
end

# Active and reactive power bounds
active_constraints = Dict{Int, ConstraintRef}()
for i in GENERATORINDEX
    @constraint(model, 0 <= active[i] <= max_capacity[i])
    @constraint(model, -0.003 * max_capacity[i] <= reactive[i] <= 0.003
* max_capacity[i])
    active_constraints[i] = @constraint(model, 0 <= active[i] <=
max_capacity[i])
end

# Power flow constraints
for k in NODEINDEX, l in NODEINDEX
    @constraint(model, p_var[k, l] == v[k]^2 * g[k, l] - v[k] * v[l] *
g[k, l] * cos(theta[k] - theta[l]) - v[k] * v[l] * b[k, l] *
sin(theta[k] - theta[l]))
    @constraint(model, q_var[k, l] == -v[k]^2 * b[k, l] + v[k] * v[l] *
b[k, l] * cos(theta[k] - theta[l]) - v[k] * v[l] * g[k, l] *
sin(theta[k] - theta[l]))
end

# Active and reactive power conservation constraints
@constraint(model, [k in NODEINDEX], sum(p_var[k, i] for i in
NODEINDEX) - G[k] + demand[k] == 0)
@constraint(model, [k in NODEINDEX], sum(q_var[k, i] for i in
NODEINDEX) - R[k] == 0)

# Generation and load balance constraints
@constraint(model, G[1] == 0)
@constraint(model, G[2] == active[1] + active[2] + active[3])
@constraint(model, G[3] == active[4])
@constraint(model, G[4] == active[5])
@constraint(model, G[5] == active[6])
@constraint(model, G[6] == 0)
@constraint(model, G[7] == active[7])
@constraint(model, G[8] == 0)
@constraint(model, G[9] == active[8] + active[9])
@constraint(model, G[10] == 0)
```

```
@constraint(model, G[11] == 0)
@constraint(model, R[1] == 0)
@constraint(model, R[2] == reactive[1] + reactive[2] + reactive[3])
@constraint(model, R[3] == reactive[4])
@constraint(model, R[4] == reactive[5])
@constraint(model, R[5] == reactive[6])
@constraint(model, R[6] == 0)
@constraint(model, R[7] == reactive[7])
@constraint(model, R[8] == 0)
@constraint(model, R[9] == reactive[8] + reactive[9])
@constraint(model, R[10] == 0)
@constraint(model, R[11] == 0)

# Solve the model
optimize!(model)

# Display results
println("\n------------")
println("solver_result: ", termination_status(model))
println("active: ", value.(active))
println("reactive: ", value.(reactive))
println("Voltage Amplitude: ", value.(v))
println("Phase angle: ", value.(theta))
println("Active Power Flow: ", value.(p_var))
println("Reactive Power Flow: ", value.(q_var))
println("Total Cost: ", objective_value(model))

# To find the generator with the most significant cost-reducing dual
value
dual_values = Dict{Int, Float64}()
for i in GENERATORINDEX
    dual_values[i] = dual(active_constraints[i])
end
optimal_generator_index = argmin(dual_values)
println("Generator $optimal_generator_index is the most beneficial for
capacity expansion.")
```

Project_data.jl file:

```
cost = [175, 100, 150, 150, 300, 350, 400, 300, 200]
max_capacity = [0.02, 0.15, 0.08, 0.07, 0.04, 0.17, 0.17, 0.26, 0.05]
demand = [0.10, 0, 0, 0.19, 0, 0.11, 0, 0.09, 0.21, 0.05, 0.04]

# Define the dimensions of the 2D array
n = 11
b = zeros(n, n)
g = zeros(n, n)

# Assign specific values to the elements
b[1, 2] = -20.1
b[2, 1] = -20.1
b[1, 11] = -22.3
b[11, 1] = -22.3
b[2, 3] = -16.8
b[3, 2] = -16.8
b[2, 11] = -17.2
b[11, 2] = -17.2
b[3, 4] = -11.7
b[4, 3] = -11.7
b[3, 9] = -19.4
b[9, 3] = -19.4
b[4, 5] = -10.8
b[5, 4] = -10.8
b[5, 6] = -12.3
b[6, 5] = -12.3
b[5, 8] = -9.2
b[8, 5] = -9.2
b[6, 7] = -13.9
b[7, 6] = -13.9
b[7, 8] = -8.7
b[8, 7] = -8.7
b[7, 9] = -11.3
b[9, 7] = -11.3
b[8, 9] = -14.7
b[9, 8] = -14.7
b[9, 10] = -13.5
b[10, 9] = -13.5
b[10, 11] = -26.7
b[11, 10] = -26.7
```

```
# Assign specific values to the elements
g[1, 2] = 4.12
g[2, 1] = 4.12
g[1, 11] = 5.67
g[11, 1] = 5.67
g[2, 3] = 2.41
g[3, 2] = 2.41
g[2, 11] = 2.78
g[11, 2] = 2.78
g[3, 4] = 1.98
g[4, 3] = 1.98
g[3, 9] = 3.23
g[9, 3] = 3.23
g[4, 5] = 1.59
g[5, 4] = 1.59
g[5, 6] = 1.71
g[6, 5] = 1.71
g[5, 8] = 1.26
g[8, 5] = 1.26
g[6, 7] = 1.11
g[7, 6] = 1.11
g[7, 8] = 1.32
g[8, 7] = 1.32
g[7, 9] = 2.01
g[9, 7] = 2.01
g[8, 9] = 2.41
g[9, 8] = 2.41
g[9, 10] = 2.14
g[10, 9] = 2.14
g[10, 11] = 5.06
g[11, 10] = 5.06
```

Appendix B: Code Output:

```
******************************************************************************
This program contains Ipopt, a library for large-scale nonlinear optimization.
 Ipopt is released as open source code under the Eclipse Public License (EPL).
         For more information visit https://github.com/coin-or/Ipopt
******************************************************************************
This is Ipopt version 3.14.13, running with linear solver MUMPS 5.6.1.
Number of nonzeros in equality constraint Jacobian...:     1106
Number of nonzeros in inequality constraint Jacobian.:       98
Number of nonzeros in Lagrangian Hessian.............:     1080

Total number of variables............................:      304
                     variables with only lower bounds:        0
                variables with lower and upper bounds:        0
                     variables with only upper bounds:        0
Total number of equality constraints.................:      286
Total number of inequality constraints...............:       98
        inequality constraints with only lower bounds:       49
   inequality constraints with lower and upper bounds:        0
        inequality constraints with only upper bounds:       49
iter    objective    inf_pr   inf_du lg(mu)  ||d||  lg(rg) alpha_du alpha_pr  ls
   0  0.0000000e+00 9.80e-01 1.00e+02  -1.0 0.00e+00    -  0.00e+00 0.00e+00   0
   1  1.6512850e+01 8.04e-01 2.95e+05  -1.0 1.08e+00    -  1.10e-01 1.63e-01h  1
   2  2.5110312e+01 7.90e-01 2.66e+05  -1.0 5.28e-01   6.0 1.02e-02 4.58e-02h  1
   3  2.5199563e+01 7.90e-01 2.65e+05  -1.0 4.82e-01   5.5 2.38e-01 5.02e-04h  1
   4  2.5379299e+01 7.89e-01 2.65e+05  -1.0 7.91e-01   5.0 6.27e-02 1.05e-03h  1
   5  3.4884267e+01 7.43e-01 2.30e+05  -1.0 7.94e-01   4.6 3.61e-02 5.92e-02h  1
   6  3.0563697e+01 7.38e-01 2.28e+05  -1.0 8.25e+00    -  3.01e-03 6.12e-03h  1
   7  3.0401248e+01 7.38e-01 2.28e+05  -1.0 4.96e+00    -  4.47e-03 5.04e-04h  1
   8  3.0126340e+01 7.35e-01 2.27e+05  -1.0 1.52e+00    -  3.93e-03 3.54e-03h  1
   9  2.8751989e+01 7.25e-01 2.23e+05  -1.0 1.90e+00    -  2.37e-03 1.42e-02f  1
iter    objective    inf_pr   inf_du lg(mu)  ||d||  lg(rg) alpha_du alpha_pr  ls
  10  2.6200917e+01 7.10e-01 2.18e+05  -1.0 2.09e+00    -  2.32e-02 2.07e-02h  1
  11  2.5848249e+01 7.09e-01 2.17e+05  -1.0 1.87e+00    -  5.38e-02 1.62e-03h  1
  12  2.4510915e+01 7.05e-01 2.16e+05  -1.0 1.96e+00    -  8.07e-04 4.61e-03h  1
  13  2.4629627e+01 7.04e-01 2.16e+05  -1.0 7.23e-01    -  1.25e-02 1.99e-03h  1
  14  2.4538894e+01 6.98e-01 2.14e+05  -1.0 1.10e+00    -  2.15e-04 8.70e-03f  1
  15  2.4516958e+01 6.98e-01 2.14e+05  -1.0 1.39e+00    -  1.44e-02 3.58e-04h  1
  16  2.2383058e+01 6.69e-01 2.02e+05  -1.0 1.35e+00    -  1.04e-02 4.11e-02f  1
  17  2.2379981e+01 6.68e-01 2.02e+05  -1.0 1.05e+00    -  6.31e-03 1.15e-03h  1
  18  2.2643640e+01 6.58e-01 1.98e+05  -1.0 9.30e-01    -  1.15e-02 1.53e-02h  1
```

```
  19  2.2817270e+01 6.56e-01 1.98e+05  -1.0 6.62e-01    -  1.69e-02 2.25e-03h  1
iter    objective    inf_pr   inf_du lg(mu)  ||d||  lg(rg) alpha_du alpha_pr  ls
  20  2.3313774e+01 6.46e-01 1.94e+05  -1.0 8.39e-01    -  1.22e-03 1.61e-02h  1
  21  2.3336602e+01 6.45e-01 1.94e+05  -1.0 7.01e-01    -  7.84e-02 4.48e-04h  1
  22  2.4808710e+01 6.33e-01 1.90e+05  -1.0 6.51e-01    -  5.01e-03 1.91e-02h  1
  23  2.5145178e+01 6.31e-01 1.89e+05  -1.0 6.37e-01    -  9.30e-03 3.66e-03h  1
  24  2.5870005e+01 6.26e-01 1.88e+05  -1.0 6.34e-01    -  2.72e-02 7.77e-03h  1
  25  2.9961765e+01 6.02e-01 1.79e+05  -1.0 6.31e-01    -  2.41e-04 3.77e-02h  1
  26  3.0004740e+01 6.02e-01 1.79e+05  -1.0 6.06e-01    -  6.11e-02 4.04e-04h  1
  27  3.0136549e+01 6.01e-01 1.79e+05  -1.0 6.04e-01    -  5.32e-04 1.14e-03h  1
  28  3.4015282e+01 5.67e-01 1.66e+05  -1.0 6.14e-01    -  6.09e-04 5.68e-02h  1
  29  3.4273101e+01 5.65e-01 1.66e+05  -1.0 5.77e-01    -  7.29e-02 3.89e-03h  1
iter    objective    inf_pr   inf_du lg(mu)  ||d||  lg(rg) alpha_du alpha_pr  ls
  30  4.5085665e+01 5.23e-01 1.50e+05  -1.0 5.73e-01    -  3.57e-04 7.45e-02h  1
  31  4.6691339e+01 5.17e-01 1.48e+05  -1.0 5.29e-01    -  1.07e-01 1.18e-02h  1
  32  4.6713818e+01 5.17e-01 1.48e+05  -1.0 5.20e-01    -  5.94e-03 1.66e-04h  1
  33  6.6830959e+01 4.48e-01 1.23e+05  -1.0 5.19e-01    -  8.12e-04 1.32e-01h  1
  34  6.7827164e+01 4.45e-01 1.22e+05  -1.0 4.50e-01    -  1.66e-01 7.45e-03h  1
  35  7.9185065e+01 4.01e-01 1.08e+05  -1.0 4.45e-01    -  1.46e-02 9.95e-02h  1
  36  8.5422521e+01 3.76e-01 1.01e+05  -1.0 4.01e-01    -  2.99e-01 6.22e-02h  1
  37  1.1677963e+02 2.58e-01 5.91e+04  -1.0 3.76e-01    -  6.27e-02 3.13e-01h  1
  38  1.2431506e+02 2.34e-01 5.33e+04  -1.0 2.59e-01    -  1.66e-01 9.38e-02h  1
  39  1.3090582e+02 2.09e-01 4.74e+04  -1.0 2.35e-01    -  7.45e-02 1.07e-01h  1
iter    objective    inf_pr   inf_du lg(mu)  ||d||  lg(rg) alpha_du alpha_pr  ls
  40  1.3288378e+02 2.02e-01 4.57e+04  -1.0 2.09e-01    -  2.62e-01 3.52e-02h  1
  41  1.4070007e+02 1.74e-01 3.90e+04  -1.0 2.02e-01    -  1.67e-02 1.39e-01h  1
  42  1.6464835e+02 8.27e-02 1.58e+04  -1.0 1.75e-01    -  2.78e-01 5.24e-01h  1
  43  1.7002608e+02 6.19e-02 1.17e+04  -1.0 8.41e-02    -  1.68e-01 2.52e-01h  1
  44  1.7769728e+02 3.31e-02 6.07e+03  -1.0 6.22e-02    -  6.15e-01 4.65e-01h  1
  45  1.7973999e+02 2.33e-02 4.26e+03  -1.0 8.03e-02    -  3.00e-01 2.95e-01h  1
  46  1.8083181e+02 1.57e-02 2.86e+03  -1.0 9.51e-02    -  5.67e-01 3.27e-01h  1
  47  1.8474841e+02 6.11e-03 1.90e+03  -1.0 1.77e-02    -  2.74e-01 5.98e-01h  1
  48  1.8584496e+02 4.60e-03 1.54e+03  -1.0 1.44e-02    -  4.49e-01 2.25e-01h  1
  49  1.9301733e+02 2.60e-04 2.07e+03  -1.0 3.57e-02    -  3.41e-01 1.00e+00h  1
iter    objective    inf_pr   inf_du lg(mu)  ||d||  lg(rg) alpha_du alpha_pr  ls
  50  1.9544877e+02 2.75e-05 1.04e+03  -1.0 1.16e-02    -  5.87e-01 9.80e-01f  1
  51  1.9515011e+02 1.82e-05 7.92e+01  -1.0 4.89e-03    -  6.99e-01 1.00e+00f  1
  52  1.9420070e+02 4.38e-05 1.74e+02  -1.0 1.15e-02    -  5.70e-01 1.00e+00f  1
  53  1.9394566e+02 3.81e-06 2.55e-01  -1.0 3.79e-03    -  1.00e+00 1.00e+00f  1
  54  1.8848686e+02 7.47e-05 2.80e+02  -2.5 2.54e-02    -  8.48e-01 1.00e+00f  1
  55  1.8668365e+02 9.02e-05 1.31e+01  -2.5 9.92e-03    -  9.32e-01 1.00e+00f  1
  56  1.8656247e+02 2.39e-06 1.17e-01  -2.5 3.68e-03    -  1.00e+00 1.00e+00h  1
```

```
  57  1.8641001e+02 2.07e-06 1.26e+01  -3.8 3.29e-03    -  9.01e-01 4.99e-01f  1
  58  1.8629844e+02 7.56e-08 2.98e-02  -3.8 3.04e-03    -  1.00e+00 1.00e+00f  1
  59  1.8630124e+02 8.06e-09 5.95e-03  -3.8 6.30e-04    -  1.00e+00 1.00e+00h  1
iter    objective    inf_pr   inf_du lg(mu)  ||d||  lg(rg) alpha_du alpha_pr  ls
  60  1.8628985e+02 4.42e-09 7.52e-02  -5.7 5.36e-04    -  9.92e-01 9.39e-01f  1
  61  1.8628929e+02 5.16e-11 7.93e-05  -5.7 9.07e-05    -  1.00e+00 1.00e+00f  1
  62  1.8628915e+02 7.78e-13 3.69e-07  -8.6 8.43e-06    -  1.00e+00 1.00e+00f  1
  63  1.8628915e+02 5.19e-15 1.16e-10  -8.6 6.45e-04    -  1.00e+00 1.00e+00h  1

Number of Iterations....: 63
                                   (scaled)                 (unscaled)
Objective...............:   4.6572287017768510e+01    1.8628914807107404e+02
Dual infeasibility......:   1.1637246551564806e-10    4.6548986206259224e-10
Constraint violation....:   5.1936536668573119e-15    5.1936536668573119e-15
Variable bound violation:   0.0000000000000000e+00    0.0000000000000000e+00
Complementarity.........:   2.5059215396598776e-09    1.0023686158639510e-08
Overall NLP error.......:   8.2124150084250901e-10    1.0023686158639510e-08

Number of objective function evaluations             = 64
Number of objective gradient evaluations             = 64
Number of equality constraint evaluations            = 64
Number of inequality constraint evaluations          = 64
Number of equality constraint Jacobian evaluations   = 64
Number of inequality constraint Jacobian evaluations = 64
Number of Lagrangian Hessian evaluations             = 63
Total seconds in IPOPT                               = 0.785

EXIT: Optimal Solution Found.

------------
solver_result: LOCALLY_SOLVED
active: 1-dimensional DenseAxisArray{Float64,1,...} with index sets:
    Dimension 1, 1:9
And data, a 9-element Vector{Float64}:
 0.004874392642096726
 0.1500000097327036
 0.08000000919811077
 0.07000000982047053
 0.040000009882535555
 0.13885595312517077
 0.0030797191136281176
 0.2536821645624077
```

```
 0.05000000979952773
reactive: 1-dimensional DenseAxisArray{Float64,1,...} with index sets:
    Dimension 1, 1:9
And data, a 9-element Vector{Float64}:
 6.000999924213923e-5
 0.0004500099992421392
 0.0002400099992421392
 0.0002100099992422685
 0.00012000999924236284
 0.000510009999242348
 0.0005100099992423029
 0.0007800099992422646
 0.00015000999924226442

Voltage Amplitude: 1-dimensional DenseAxisArray{Float64,1,...} with index sets:
    Dimension 1, 1:11
And data, a 11-element Vector{Float64}:
 1.0191324516600144
 1.0200000100732174
 1.0194870880914335
 1.0181427553743867
 1.0188682855322975
 1.018212960671243
 1.018527913312949
 1.0185161290965272
 1.019219338075308
 1.0190113615479002
 1.0192297933346912

Phase Angle: 1-dimensional DenseAxisArray{Float64,1,...} with index sets:
    Dimension 1, 1:11
And data, a 11-element Vector{Float64}:
  0.0024507602713311535
  0.006909708258983965
  0.0036546928246491968
 -0.0041982870025931264
  0.00033846521617767703
 -0.0046539893008275225
 -0.001544417744648192
 -0.001914807979200308
  0.0022120630183163946
  0.0012873573864556024
```

0.0025660838784659808
Active Power Flow: 2-dimensional DenseAxisArray{Float64,2,...} with index sets:
Dimension 1, 1:11
Dimension 2, 1:11
And data, a 11×11 Matrix{Float64}:
0.0 -0.09676623272932701 0.0 0.0 0.0 0.0 0.0 0.0 0.0
0.0 -0.0032337672706729934 0.09685448514207184 0.0 0.05813893540217554 0.0 0.0 0.0 0.0 0.0 0.0
0.0 0.0798809910286637 0.0 -0.05811174890352575 -1.7650232885655612e-22 0.09814581308304221 -1.4518271674499357e-33 9.62964972193618e-35 4.988246357955311e-35 9.62964972193618e-35 0.029965945640954065 5.813406324623861e-35 5.4141647181268515e-34 2.7322422854753266e-20 -4.940215070744118e-26 -0.09801549224986307 0.0 -0.051984497867601356 8.290122654002648e-34 5.187901515549317e-35 -1.8214642610924042e-33 5.187901679545849e-35 1.546449631491616e-33 5.187901515549317e-35
0.0 9.922422374705338e-21 0.0 0.052019282757940766 6.956759721614617e-21 0.06486894094126935 0.0 0.021967729425960664 0.0 0.0 0.0 -5.36515310913322e-22 0.0 0.0 0.0 -0.06482399051386559 0.0 -0.045176009486134414 3.683895210801251e-36 0.0 2.674778638679953e-35 0.0
0.0 0.0 0.0 0.0 0.0 0.045187250621285494 0.0 0.0033588082947570053 -0.045466339802414386 0.0 0.0 1.3789912840388245e-20 0.0 0.0 0.0 -0.021960934453242133 0.0 -0.0033586202516632077 0.0 -0.06468044529509465 -1.352351322625226e-34 0.0
0.0 0.0 -0.029958729158848597 0.0 0.0 0.0 0.04549674483249498 0.06472424530502016 0.0 0.013419913383268964 0.0 -1.515797021830289e-20 0.0 0.0 0.0 0.0 0.0 0.0 0.0 0.0 -0.013417920321150408 0.0 -0.03658207967884959
0.003233899325137097 -0.0798248136685298 6.7471983028914356e-34 -1.3728633962419267e-34 -2.325068829018657e-34 -8.379198464388924e-38 -9.629777833252602e-35 5.934780006146797e-36 -4.837186665096606e-34 0.0365909143433927 4.2221964835507454e-20

Reactive Power Flow: 2-dimensional DenseAxisArray{Float64,2,...} with index sets:

Dimension 1, 1:11
Dimension 2, 1:11
And data, a 11×11 Matrix{Float64}:
0.0 0.001532886580480279 0.0 0.0 0.0
0.0 0.0 0.0 0.0
0.0 -0.001532886580480279
-0.0011023347610187226 0.0 0.0007245889985586249 0.0 0.0
0.0 0.0 0.0 0.0
0.0 0.0011277757601865154
0.0 -0.000535073157345428 0.0 0.0002703315846948068 0.0
2.7348453697056596e-20 0.0 -2.95696882823361e-21
0.00047475157189288963 2.8627806208111155e-33 0.0
0.0 0.0 0.000499746065909549 0.0 -
0.0003797360666671861 -5.314618604594253e-20 -3.239952436413086e-28 -
3.4971674259140578e-22 -4.904502543659457e-37 -2.6330737718073512e-36 0.0
0.0 0.0 0.0 0.0006160107935030996 0.0 -
0.0004849541644316594 0.0 0.0003789533701709077 -
4.906775945226744e-35 1.0882499462411036e-34 0.0
3.0488782415414396e-20 0.0 0.0 -2.1969717631912185e-36
0.0008082818001415952 0.0 -0.0008082818001415953 -
3.7092061506874214e-68 0.0
-1.2054704789444919e-52 0.0
0.0 0.0 0.0 0.0 7.07107146756688e-21
0.0009490491682495163 0.0 -0.00040215292219906773 -
3.688624680814571e-5 -1.1138527892964124e-35 0.0
-1.2865485505927174e-20 0.0 0.0 0.0 -
0.0003293392836518634 0.0 0.00040339229713354805 0.0 -
7.405301348168467e-5 2.692573889305832e-35 0.0
0.0 -3.2436340813288356e-28 -0.000431407995164976
4.901048328535402e-36 -1.819655057430169e-35 -1.588961617209962e-35
0.00020781999801071718 0.0003412148997495847 4.259170865505167e-22
0.0008123930958892033 2.910274260131983e-35
-1.153033172510297e-19 0.0 0.0 0.0 0.0
0.0 0.0 0.0 -0.0007998200404702327 0.0
0.0007998200404702329
0.0015334059481308935 -0.0007802036039611912 -4.1843309425639063e-35
1.4209055022655566e-35 3.434422355086893e-35 -9.052565771535228e-35
8.469614475910241e-35 -2.9197407424800413e-35 -2.2220113811465873e-35 -
0.0007532023441697024 6.781150077785013e-20

Total Cost: 186.28914807107404
Generator 5 is the most beneficial for capacity expansion.

Appendix C: Results (2D DenseAxisArray)

Table 4. Active power matrix for nodes (11x11)

	1	2	3	4	5	6	7	8	9	10	11
1	0	-0.0968	0	0	0	0	0	0	0	0	-0.0032
2	0.0969	0	0.0581	0	0	0	0	0	0	0	0.0799
3	0	-0.0581	0	0.0981	0	-4.5E-23	0	7.6E-23	0.0300	-2.7E-34	0
4	0	0	-0.0980	0	-0.0520	-2.7E-23	8.9E-24	-2.2E-23	-4.4E-38	1.3E-23	0
5	0	0	0	0.0520	0	0.06487	0	0.0220	0	0	0
6	-2.0E-23	0	0	0	-0.0648	0	-0.0452	8.9E-35	0	-2.4E-35	0
7	0	0	0	0	0	0.04519	0	0.00336	-0.0455	0	0
8	-1.3E-23	0	0	0	-0.0220	0	-0.0034	0	-0.0647	6.0E-34	0
9	0	-2E-23	-0.0300	0	0	0	0.0455	0.06472	5.9E-23	1.3E-02	0
10	4.0E-23	0	0	0	0	0	0	0	-0.0134	0	-0.0366
11	0.0032	-0.0798	8.7E-34	-8.8E-34	5.1E-34	8.6E-34	-1.4E-33	1.4E-33	-1.3E-33	0.0366	0.0000

Table 5. Reactive power matrix for nodes (11x11)

	1	2	3	4	5	6	7	8	9	10	11
1	0.0E+00	0.0015	0	0	0	0	0	0	0	0	-0.0015
2	-0.0011	0	0.0007	0	0.0E+00	0	0	0	0.0E+00	0	0.0011
3	0	-0.0005	0	0.0003	0	-2.0E-22	0	-4.9E-25	0.0005	-2.8E-31	0
4	0	0	0.0005	0	-0.0004	-3.1E-22	-2.3E-25	-2.7E-26	-1.9E-36	-1.9E-35	0
5	0	0	0	0.0006	0	-0.0005	0	0.0004	2.4E-35	2.4E-30	0
6	1.3E-22	0	0	0	0.0008	0	-0.0008	-7.5E-37	0	-7.5E-37	0
7	0	0	0	0	-2.1E-22	0.0009	0	-0.0004	-3.7E-05	-6.4E-37	0
8	-1.4E-22	0	0.0E+00	0.0E+00	-3.3E-04	0.0E+00	4.0E-04	0.0E+00	-7.4E-05	-2.1E-37	-1.0E-41
9	0	-4.53E-25	-0.0004	-3.9E-35	-7.4E-35	4.2E-36	0.0002	0.0003	-1.8E-22	0.0008	-6.5E-36
10	-1.8E-21	0	0	0	0	0	0	1.8E-36	-0.0008	0	0.0008
11	0.0015	-0.0008	1.80E-35	-4.6E-35	1.3E-35	2.8E-35	-2.1E-35	-1.9E-35	1.7E-35	-0.0008	-3.8E-22